



Yıldız CTI Teknik Raporu

THOR SCRAPER PROJESİ

Furkan YOLCU

İçindekiler

1. GİRİŞ VE AMAÇ	3
2. KULLANILAN TEKNOLOJİLER	3
2.1 Programlama Dili	3
2.2 Harici Kütüphaneler	3
2.3 Go Standart Kütüphaneleri	3
3. PROGRAM MİMARİSİ VE ÇALIŞMA MANTIĞI	4
3.1 Sabit Değerler ve Yapılandırma	4
3.2 User-Agent Kullanımı	5
3.3 Timeout Yönetimi	5
3.4 Goroutine ile Paralel Tarama	6
Goroutine Yapısının Açıklaması:	7
Semaphore Mekanizması:	7
3.5 Temel Fonksiyonlar	8
3.6 Program Akışı	8
4. KULLANIM KILAVUZU	9
4.1 Gereksinimler	9
4.2 Kurulum	9
4.3 Çalıştırma	9
5. HATA YÖNETİMİ	9
6. TEST EDİLEN WEB SİTELERİ	9
7. OLUŞTURULAN DOSYA YAPISI	10
8. EK ÖZELLİKLER	10
8.1 URL Listeleme	10
8.2 Yapılandırılabilir Timeout	10
8.3 JSON Rapor Formatı	11
8.4 HTTP Durum Kodu Kontrolü	11
9. ÖRNEK ÇALIŞTIRMA ÇIKTILARI	11
9.2 Ekran Görüntüsü Örneği	Hata! Yer işareti tanımlanmamış.
10. SONUÇ	13

1. GİRİŞ VE AMAÇ

Bu proje, Tor ağı üzerinden .onion uzantılı web sitelerine erişim sağlayarak içerik tarama, HTML kaydetme ve ekran görüntüsü alma işlemlerini gerçekleştiren bir araçtır. Program, Go programlama dili kullanılarak geliştirilmiş olup, paralel işlem (goroutine) desteği ile yüksek performanslı tarama yapabilmektedir.

Projenin temel amaçları:

- Tor proxy üzerinden güvenli bağlantı kurma
- Hedef web sitelerinden HTML içerik çekme
- Otomatik ekran görüntüsü alma
- Sayfa içi linkleri çıkarma
- JSON formatında tarama raporu oluşturma

2. KULLANILAN TEKNOLOJİLER

2.1 Programlama Dili

- **Go (Golang) 1.21+:** Yüksek performanslı, derlenebilir ve concurrency desteği güçlü olan Go dili tercih edilmiştir. Go'nun goroutine yapısı sayesinde paralel HTTP istekleri verimli şekilde yönetilmektedir.

2.2 Harici Kütüphaneler

Kütüphane	Açıklama
github.com/chromedp/chromedp	Headless Chrome kontrolü için kullanılır. Ekran görüntüsü alma işlemlerinde chromedp paketi ile tarayıcı otomasyonu sağlanır.
golang.org/x/net/proxy	SOCKS5 proxy bağlantısı kurmak için kullanılır. Tor ağına bu paket aracılığıyla bağlanılır.

2.3 Go Standart Kütüphaneleri

Kütüphane	Kullanım Amacı
net/http	HTTP istemci oluşturma ve GET istekleri gönderme
crypto/tls	TLS yapılandırması ve sertifika doğrulama ayarları

Kütüphane	Kullanım Amacı
encoding/json	Tarama sonuçlarını JSON formatında kaydetme
context	Timeout yönetimi ve iptal mekanizması
sync	WaitGroup ile goroutine senkronizasyonu
regexp	HTML içerikten link çıkarma işlemleri
bufio	Dosya okuma ve kullanıcı girdi işlemleri
io	HTTP yanıt gövdesi okuma
os	Dosya ve klasör işlemleri
filepath	Platform bağımsız dosya yolu oluşturma
time	Zaman damgası ve timeout süreleri
strings	Metin işleme ve parse etme
fmt	Formatlı çıktı ve loglama

3. PROGRAM MİMARİSİ VE ÇALIŞMA MANTIĞI

3.1 Sabit Değerler ve Yapılandırma

```
1  const (  
2      TorProxyAddr    = "127.0.0.1:9050"  
3      TorBrowserProxy = "127.0.0.1:9150"  
4      RequestTimeout  = 45 * time.Second  
5      OutputDir       = "output"  
6      ScreenshotDir   = "screenshots"  
7      LogsDir         = "logs"  
8      ReportFile      = "scan_report.log"  
9      UserAgent        = "Mozilla/5.0 (Windows NT 10.0; rv:128.0) Gecko/20100101  
        Firefox/128.0"  
10 )
```

3.2 User-Agent Kullanımı

Program, HTTP isteklerinde Tor Browser'ın kullandığı User-Agent değerini kullanmaktadır:

```
1 UserAgent = "Mozilla/5.0 (Windows NT 10.0; rv:128.0) Gecko/20100101 Firefox/128.0"
```

Bu User-Agent değeri, her HTTP isteğinde header olarak gönderilir:

```
1 req.Header.Set("User-Agent", UserAgent)
2 req.Header.Set("Accept",
3   "text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8")
4 req.Header.Set("Accept-Language", "en-US,en;q=0.5")
5 req.Header.Set("Connection", "keep-alive")
```

User-Agent kullanımının önemi:

- Hedef sunuculara gerçek bir tarayıcıdan geliyormuş gibi görünme
- Bot tespiti mekanizmalarını atlatma
- Tor Browser ile tutarlı parmak izi oluşturma
- Bazı sitelerin User-Agent kontrolünü geçme

3.3 Timeout Yönetimi

Program iki seviyede timeout kontrolü uygulamaktadır:

1. HTTP Client Timeout (45 saniye):

```
1 RequestTimeout = 45 * time.Second
2
3 client := &http.Client{
4     Transport: transport,
5     Timeout:   RequestTimeout,
6     CheckRedirect: func(req *http.Request, via []*http.Request) error {
7         if len(via) >= 10 {
8             return fmt.Errorf("çok fazla yönlendirme")
9         }
10        return nil
11    },
12 }
```

2. Screenshot Timeout (45 saniye):

```
1 ctx, cancel = context.WithTimeout(ctx, 45*time.Second)
2 defer cancel()
```

Timeout mekanizmasının avantajları:

- Yanıt vermeyen sitelerde programın takılmasını önleme
- Kaynak kullanımını optimize etme
- Tor ağındaki yavaş bağlantıları yönetme
- Kullanıcı deneyimini iyileştirme

3.4 Goroutine ile Paralel Tarama

Program, birden fazla siteyi aynı anda taramak için Go'nun goroutine yapısını kullanmaktadır. Bu sayede tarama süresi önemli ölçüde kısalmaktadır.

```
1 func ScanForums(client *http.Client, forums []Forum, logger *Logger) *ScanReport {
2     var wg sync.WaitGroup
3     resultChan := make(chan ScanResult, len(forums))
4
5     semaphore := make(chan struct{}, 5)
6
7     for _, forum := range forums {
8         wg.Add(1)
9         go func(f Forum) {
10             defer wg.Done()
11
12             semaphore <- struct{}{}
13             defer func() { <-semaphore }()
14
15             result := ScanForum(client, f, logger)
16             resultChan <- result
17         }(forum)
18     }
19
20     go func() {
21         wg.Wait()
22         close(resultChan)
23     }()
24
25     for result := range resultChan {
26         report.Results = append(report.Results, result)
27     }
28
29     return report
30 }
```

```
[2025-12-29 18:33:41] [INFO] =====
[2025-12-29 18:33:41] [INFO] TARAMA TAMAMLANDI - Search Engines
[2025-12-29 18:33:41] [INFO] =====
[2025-12-29 18:33:41] [SUCCESS] Başarılı: 3
[2025-12-29 18:33:41] [ERR] Başarısız: 0
[2025-12-29 18:33:41] [INFO] Süre: 16.96 saniye
```

Üstteki ekran görüntüsünde goroutine ile paralel tarama, alttaki görüntüde ise sıralı basit bir for döngüsü ile yapılan tarama çıktısı yer almaktadır. Görüldüğü üzere, goroutine kullanımı sayesinde birden fazla site eş zamanlı taranarak toplam işlem süresi önemli ölçüde azaltılmıştır. Bu durum, Paralel tarama yapmanın Tor ağı gibi yüksek gecikmeli ortamlarda sağladığı performans avantajını kanıtlamaktadır.

```
[2025-12-29 18:25:41] [INFO] =====
[2025-12-29 18:25:41] [INFO] TARAMA TAMAMLANDI - Search Engines
[2025-12-29 18:25:41] [INFO] =====
[2025-12-29 18:25:41] [SUCCESS] Başarılı: 3
[2025-12-29 18:25:41] [ERR] Başarısız: 0
[2025-12-29 18:25:41] [INFO] Süre: 57.42 saniye
```

Goroutine Yapısının Açıklaması:

Bileşen	Açıklama
sync.WaitGroup	Tüm goroutine'lerin tamamlanmasını beklemek için kullanılır
resultChan	Tarama sonuçlarını toplamak için buffered channel
semaphore	Aynı anda maksimum 5 goroutine çalışmasını sağlayan rate limiter
go func()	Her forum için yeni bir goroutine başlatılır

Semaphore Mekanizması:

```
1 semaphore := make(chan struct{}, 5)
2
3 semaphore <- struct{}{}
4 defer func() { <-semaphore }()
```

Bu yapı sayesinde:

- Aynı anda maksimum 5 HTTP isteği yapılır
- Tor proxy'nin aşırı yüklenmesi önlenir
- Sistem kaynakları verimli kullanılır
- Rate limiting ile hedef sunucular korunur

3.5 Temel Fonksiyonlar

Fonksiyon	Açıklama
CreateTorClient()	SOCKS5 proxy üzerinden Tor bağlantısı kurar
VerifyTorConnection()	Tor API'sine istek atarak bağlantıyı doğrular
ReadTargets()	YAML formatındaki hedef dosyasını okur ve parse eder
ScanForum()	Tek bir forumu tarar, HTML kaydeder, screenshot alır
ScanForums()	Goroutine ile paralel tarama yapar
ExtractLinks()	HTML içerikten regex ile linkleri çıkarır
TakeScreenshot()	Headless Chrome ile ekran görüntüsü alır
SaveHTML()	HTML içeriği dosyaya kaydeder
SaveReport()	JSON formatında tarama raporu oluşturur

3.6 Program Akışı

1. Program başlatılır
2. Çıktı klasörleri oluşturulur (output, screenshots, logs)
3. Logger başlatılır
4. Tor proxy bağlantısı kurulur (önce 9150, sonra 9050 portu denenir)
5. Tor bağlantısı check.torproject.org API'si ile doğrulanır
6. targets.yaml dosyası okunur ve kategorilere ayrılır
7. Kullanıcıya menü gösterilir
8. Seçilen siteler goroutine ile paralel taranır
9. Her site için HTML kaydedilir ve screenshot alınır
10. Sonuçlar JSON raporu olarak kaydedilir

4. KULLANIM KILAVUZU

4.1 Gereksinimler

- Go 1.21 veya üzeri
- Tor Browser veya Tor Service
- Google Chrome (screenshot için)

4.2 Kurulum

```
1 go mod tidy
2 go build -o thor-scrapers.exe
```

4.3 Çalıştırma

```
1 # Varsayılan targets.yaml dosyası ile
2 ./thor-scrapers.exe
3
4 # Özel hedef dosyası ile
5 ./thor-scrapers.exe custom_targets.yaml
```

5. HATA YÖNETİMİ

Program aşağıdaki hata durumlarını yönetmektedir:

Hata Türü	Açıklama	Yönetim
Timeout	Site yanıt vermediğinde	45 saniye sonra bağlantı kesilir
Connection Refused	Site erişilemez durumda	Hata loglanır, sonraki siteye geçilir
TLS Hatası	Sertifika sorunları	InsecureSkipVerify ile atlanır
Yönlendirme Döngüsü	Sonsuz redirect	Maksimum 10 yönlendirme limiti

6. TEST EDİLEN WEB SİTELERİ

Program aşağıdaki kategorilerdeki .onion siteleri ile test edilmiştir:

- **Arama Motorları:** DuckDuckGo, Ahmia, Torch, Excavator

- **Haber Siteleri:** BBC News, The New York Times, ProPublica
- **Forumlar:** Hidden Answers, Dread, Suprbay, Ramble, Pitch
- **Sosyal Medya:** Facebook Onion
- **Devlet Kurumları:** CIA Onion, NCIDE
- **Alışveriş Siteleri:** Rolex

7. OLUŞTURULAN DOSYA YAPISI

```
1  thor-scrapers/  
2  ├── main.go  
3  ├── go.mod  
4  ├── go.sum  
5  ├── targets.yaml  
6  ├── output/  
7  |   ├── DuckDuckGo_20241231_120000.html  
8  |   ├── BBC_News_20241231_120100.html  
9  |   └── scan_results.json  
10 ├── screenshots/  
11 |   ├── DuckDuckGo_20241231_120000.png  
12 |   └── BBC_News_20241231_120100.png  
13 └── logs/  
14     └── scan_report.log
```

8. EK ÖZELLİKLER

8.1 URL Listeleme

Program, taranan sayfalardan .onion uzantılı linkleri otomatik olarak çıkarır ve raporlar:

```
1 func ExtractLinks(htmlContent []byte, baseURL string) []string {  
2     linkRegex := regexp.MustCompile(`href=["']([^\"]+)[\"']`)  
3     // .onion veya http ile başlayan linkler filtrelenir  
4 }
```

8.2 Yapılandırılabilir Timeout

Timeout değeri sabit olarak 45 saniye tanımlanmıştır. Bu değer, Tor ağının yavaşlığı göz önünde bulundurularak optimize edilmiştir.

8.3 JSON Rapor Formatı

```
{
  "start_time": "2025-12-31T17:42:45.2992781+03:00",
  "end_time": "2025-12-31T17:43:22.6611242+03:00",
  "total_urls": 1,
  "success_count": 1,
  "fail_count": 0,
  "results": [
    {
      "url": "http://rolex7cnn5czdjlfhtxjkxbf2ysvodhxpregxgna6vdesx5oweg4uwyd.onion",
      "status": "SUCCESS",
      "timestamp": "2025-12-31T17:42:45.2992781+03:00",
      "file_path": "output\\Rolex_20251231_174252.html",
      "screenshot_path": "screenshots\\Rolex_20251231_174322.png",
      "links_extracted": [
        "http://rolex7cnn5czdjlfhtxjkxbf2ysvodhxpregxgna6vdesx5oweg4uwyd.onion"
      ]
    }
  ]
}
```

8.4 HTTP Durum Kodu Kontrolü

Her HTTP yanıtının durum kodu kontrol edilir ve başarı/hata durumu buna göre belirlenir.

9. ÖRNEK ÇALIŞTIRMA ÇIKTILARI

Programın çalışma anında konsola yazdırılan log çıktıları aşağıdaki ekran görüntüsünde gösterilmektedir. Her işlem için zaman damgası, durum seviyesi (INFO, SUCCESS, ERR) ve açıklama mesajı yer almaktadır.

```
PS C:\Users\Furkan\Desktop\thor-scrapers> go run main.go
[2025-12-31 17:37:12] [INFO] Yeni oturum başlatıldı
[2025-12-31 17:37:12] [INFO] Program başlatıldı
[2025-12-31 17:37:12] [INFO] Tor bağlantı kontrolü ediliyor (Port: 9150)...
[2025-12-31 17:37:12] [INFO] Tor proxy test ediliyor: 127.0.0.1:9150
[2025-12-31 17:37:12] [SUCCESS] Tor proxy bağlantısı başarılı: 127.0.0.1:9150
[2025-12-31 17:37:14] [SUCCESS] Tor bağlantısı başarılı
[2025-12-31 17:37:14] [INFO] Hedef dosyası: targets.yaml
[2025-12-31 17:37:14] [INFO] targets.yaml okundu - 7 kategori, 18 URL yüklendi
[2025-12-31 17:37:14] [INFO] Kategori: Search Engines (4 site)
[2025-12-31 17:37:14] [INFO] Kategori: News Sites (3 site)
[2025-12-31 17:37:14] [INFO] Kategori: Forums (4 site)
[2025-12-31 17:37:14] [INFO] Kategori: Social Media (2 site)
[2025-12-31 17:37:14] [INFO] Kategori: Government (2 site)
[2025-12-31 17:37:14] [INFO] Kategori: Marketplace (1 site)
[2025-12-31 17:37:14] [INFO] Kategori: Dead Sites - Test (2 site)
[2025-12-31 17:37:14] [SUCCESS] Scan raporu oluşturuldu: logs\scan_report.log
```

=== Dark Web Forum Scraper ===

--- Kategoriler ---

1. Search Engines (4 site)
2. News Sites (3 site)
3. Forums (4 site)
4. Social Media (2 site)
5. Government (2 site)
6. Marketplace (1 site)
7. Dead Sites - Test (2 site)
8. Scrape ALL sites
0. Exit

Kategori seçiniz: █

=== Dark Web Forum Scraper ===

--- Kategoriler ---

1. Search Engines (4 site)
2. News Sites (3 site)
3. Forums (4 site)
4. Social Media (2 site)
5. Government (2 site)
6. Marketplace (1 site)
7. Dead Sites - Test (2 site)
8. Scrape ALL sites
0. Exit

Kategori seçiniz: 6

=== Dark Web Forum Scraper ===

--- Marketplace ---

1. Rolex
2. Scrape all in this category
0. Back to categories

Site seçiniz: 1

```
[2025-12-31 17:42:45] [INFO] Scraping: Rolex
[2025-12-31 17:42:45] [INFO] URL: http://rolex7cnn5czdjlftxjkb2ysvodhxpregxgna6vdesx5oweg4uwyd.onion
[2025-12-31 17:42:45] [INFO] Forum scraping başlatıldı: Rolex (http://rolex7cnn5czdjlftxjkb2ysvodhxpregxgna6vdesx5oweg4uwyd.onion)
[2025-12-31 17:42:52] [SUCCESS] Successfully navigated to: Rolex
[2025-12-31 17:42:52] [SUCCESS] Screenshot captured for Rolex (13671 bytes)
[2025-12-31 17:42:52] [SUCCESS] Link extracted: http://rolex7cnn5czdjlftxjkb2ysvodhxpregxgna6vdesx5oweg4uwyd.onion
[2025-12-31 17:42:52] [SUCCESS] Link saved for Rolex
[2025-12-31 17:42:52] [INFO] Screenshot alınıyor: Rolex
[2025-12-31 17:42:52] [INFO] Lütfen bekleyiniz... (45 saniye kadar sürebilir)
[2025-12-31 17:43:22] [SUCCESS] Screenshot kaydedildi: screenshots\Rolex_20251231_174322.png
[2025-12-31 17:43:22] [SUCCESS] Scraping başarılı: Rolex - Screenshot: screenshots\Rolex_20251231_174322.png
[2025-12-31 17:43:22] [SUCCESS] Scan güncellendi: Rolex - (37.36 saniye)
```

devam etmek için Enter'a basın...█

10. SONUÇ

Bu projede Go programlama dilinin güçlü concurrency özellikleri kullanılarak verimli bir Tor tarayıcı geliştirilmiştir. Goroutine yapısı sayesinde çoklu site taraması paralel olarak gerçekleştirilmekte, semaphore mekanizması ile rate limiting uygulanmaktadır. User-Agent header'ı ile gerçek bir tarayıcı taklit edilmekte, 45 saniyelik timeout ile yanıt vermeyen siteler için kaynak israfı önlenmektedir.



